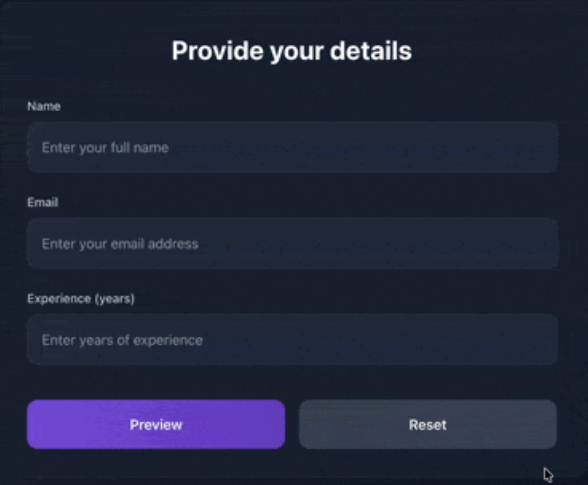


Question - 1

React: Job Application Portal

In this application, the task is to build a Job Application Portal where users can fill out and submit their job applications. Edit the *App.js*, *JobApplicationForm.js*, *Preview.js* and *SuccessMessage.js* to ensure the application functions as described below:

[Hide animation](#)



The image shows a dark-themed web form titled "Provide your details". It contains three input fields: "Name" with a placeholder "Enter your full name", "Email" with a placeholder "Enter your email address", and "Experience (years)" with a placeholder "Enter years of experience". Below the fields are two buttons: a purple "Preview" button and a grey "Reset" button. The form is set against a dark blue background.

Functionality Requirements:

1. Initial State of the project:
 - The form should have input fields for:
 - Name (required)
 - Email (required, valid email format) : Ex: test@email.com, is a valid email.
 - Experience (required, numeric input).
 - Input fields must be initially empty.
2. Preview Functionality:
 - After filling the form, users can click the Preview button to review their inputs.
 - The preview should display the entered name, email, and experience.
3. Clear Functionality:
 - In the preview, clicking the Clear button should take the user back to the form, with all the previously entered values cleared.
 - Validation:
 - Display the following error message if:
 - Name is empty: "Name is required."
 - Email is empty or invalid: "Enter a valid email address."
 - Experience is not a positive integer: "Experience must be a positive number."
 - The Preview button should not perform any operation if any field is empty or invalid.
 - Submission:
 - Upon clicking the Submit button in the preview, show a success message indicating that the application was successfully submitted.
 - Reset and Home Functionality:
 - A Reset button in the form and Home button in the success message:
 - Should clear all inputs and error messages.
 - Return the user to the initial state of the application.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

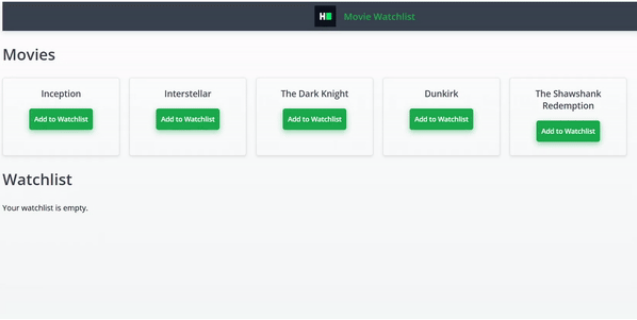
data-testid	Description
app	container representing entire application
job-application-form	The form where users can enter their application details.
input-name	Input box to enter the applicant's name.
error-name	Displays the error message related to name.
input-email	Input box to enter the applicant's email address.
error-email	Displays the error message related to email.
input-experience	Input box to enter the applicant's years of experience.
error-experience	Displays the error message related to experience.
preview-button	Button to preview the application data.
reset-button	Button to reset the form or success message.
preview	The preview section displaying the entered details.
preview-name	The preview text displaying the entered name.
preview-email	The preview text displaying the entered email.
preview-experience	The preview text displaying the entered experience.
clear-button	Button in the preview to return to the form.
submit-button	Button to submit the application from the preview.
success-message	The message displayed upon successful submission of the form.

Question - 2

React: Movie Watchlist

In this application, the task is to manage a user's movie watchlist. Edit the *Watchlist.js*, *WatchlistApp.js*, and *MovieCard.js* components within the project structure to ensure the application functions as described below:

Hide animation



Functionality Requirements:

- Initial State of the project:
 - All movies should be displayed as cards in the "Movies" section, each with an "Add to Watchlist" button. The *MovieCard* component is used to generate a card for movie.
 - The watchlist section should be empty initially, displaying a message: "Your watchlist is empty."
 - The "Add to Watchlist" button should be enabled for all movies by default.

2. Watchlist Addition Functionality:
 - When the "Add to Watchlist" button is clicked on a movie, the movie should be added to the watchlist.
 - The "Add to Watchlist" button should be disabled for movies already in the watchlist, and it's text should be changed to "Added".
3. Watchlist Display Functionality:
 - The watchlist should display movies as cards, similar to the main movie section, with each card containing a "Remove" button, using the same MovieCard component.
 - Clicking the "Remove" button should remove the movie from the watchlist, and the corresponding "Add to Watchlist" button should be re-enabled.
4. Clear Watchlist Functionality:
 - A "Clear Watchlist" button should be displayed in the watchlist section if there are movies in the watchlist.
 - Clicking the "Clear Watchlist" button should remove all movies from the watchlist and display the message: "Your watchlist is empty."
5. Movie Count Display:
 - The total number of movies in the watchlist should be displayed in a badge/icon near the watchlist title. For example: "Watchlist (2)".

Note:
The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

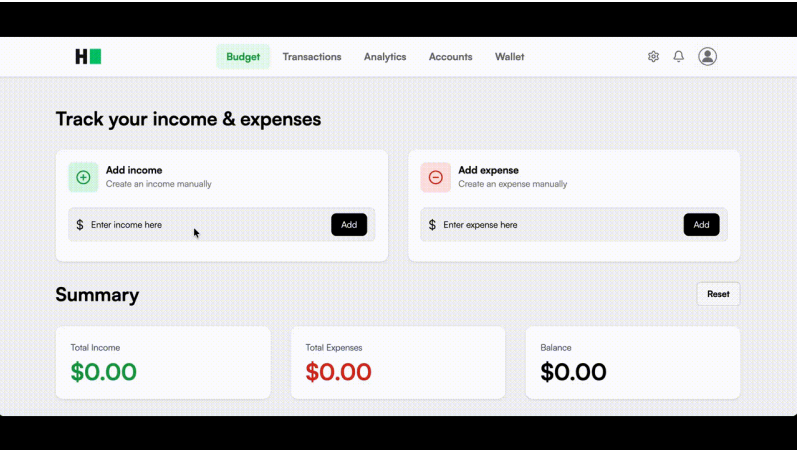
Table of data-testid

data-testid	Description
movie-card-{id}	Movie card displaying the movie alongwith Remove button.
movie-title-{id}	The title of the movie displayed in either the movie list or watchlist.
add-btn-{id}	Button to add the movie with the specified ID to the watchlist.
remove-btn-{id}	Button to remove the movie with the specified ID from the watchlist.
watchlist-container	Displays all the movies in the watchlist alongwith its count.
watchlist-empty	Message indicating that the watchlist is empty.
movie-container	Displays all the movies in the watchlist.
clear-watchlist-btn	Button to clear all movies from the watchlist.

Question - 3
React: Budget Tracker

Build a budget tracker that allows users to manage their income and expenses effectively. The application should provide a clear view of total income, total expenses, and the remaining balance.

[Hide animation](#)



Your users should be able to:

- Enter income or expense amounts in input fields.
- Add income values to the total income, with the balance updating correctly.
- Add expense values to the total expenses, with the balance updating correctly.
- See the total income, total expenses, and remaining balance at all times.
- Provide only positive numbers as input. Zero, negative, or empty inputs should not change totals or balance.
- Clear input fields automatically after adding valid income or expenses.
- Reset the tracker using a Reset button, which sets total income, expenses, and balance back to 0.00 and restores the initial state.

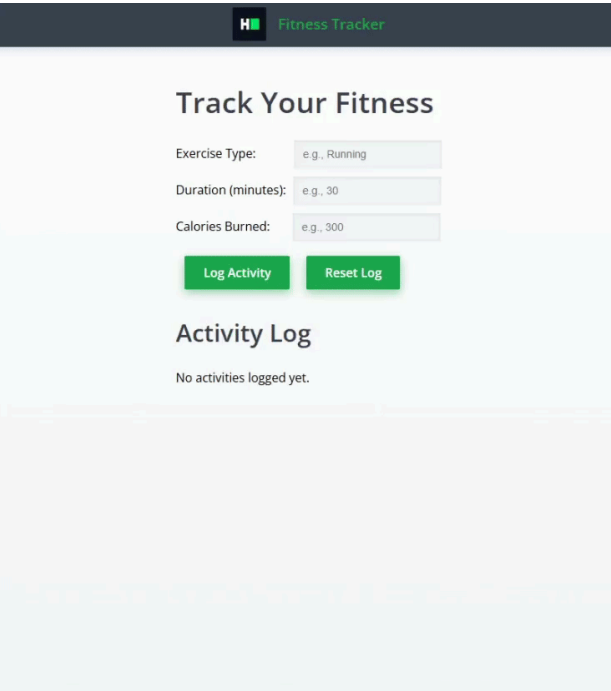
Note: Refer to the README.md file for detailed implementation requirements.

Question - 4

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)



Functionality Requirements:

1. Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
2. Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
3. Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.

- Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
4. Error Handling:
- Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.
 - "Exercise type must not be empty." : If the exercise type input is empty.
 - "Duration must be a positive number.": If the duration input is empty or non-positive.
 - "Calories must be a positive number.": If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 5

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
 - The image itself when its visible attribute is true.
 - The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div

<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 1

SCORE: 50 points

React: Customer Search

React Hooks

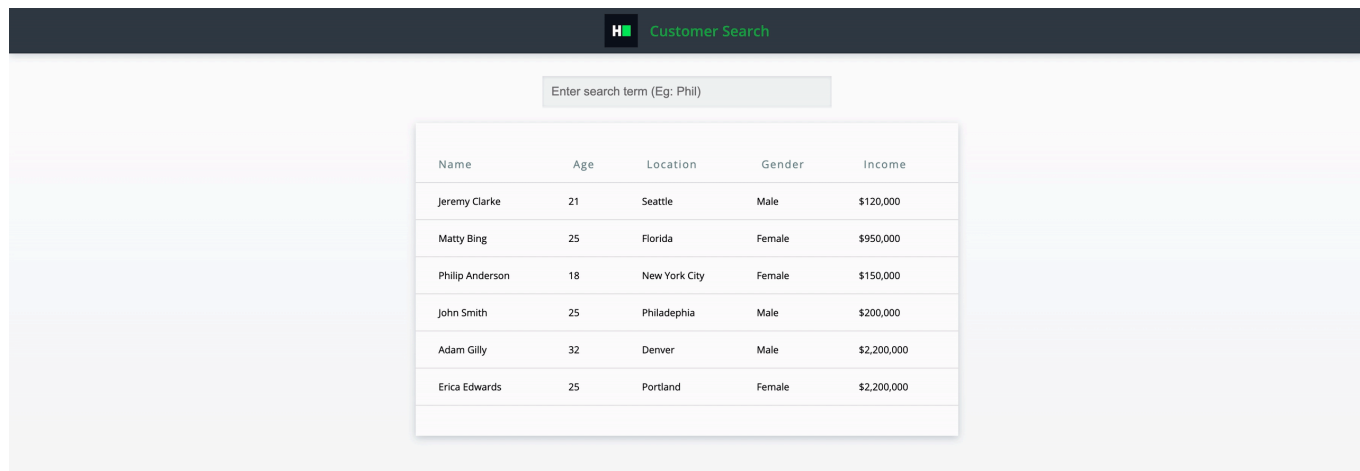
React Component

Easy

React States

JavaScript

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)

The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
  income: "$120,000"
}
```

The application has 2 components:

- `SearchCustomer` - contains a search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 2

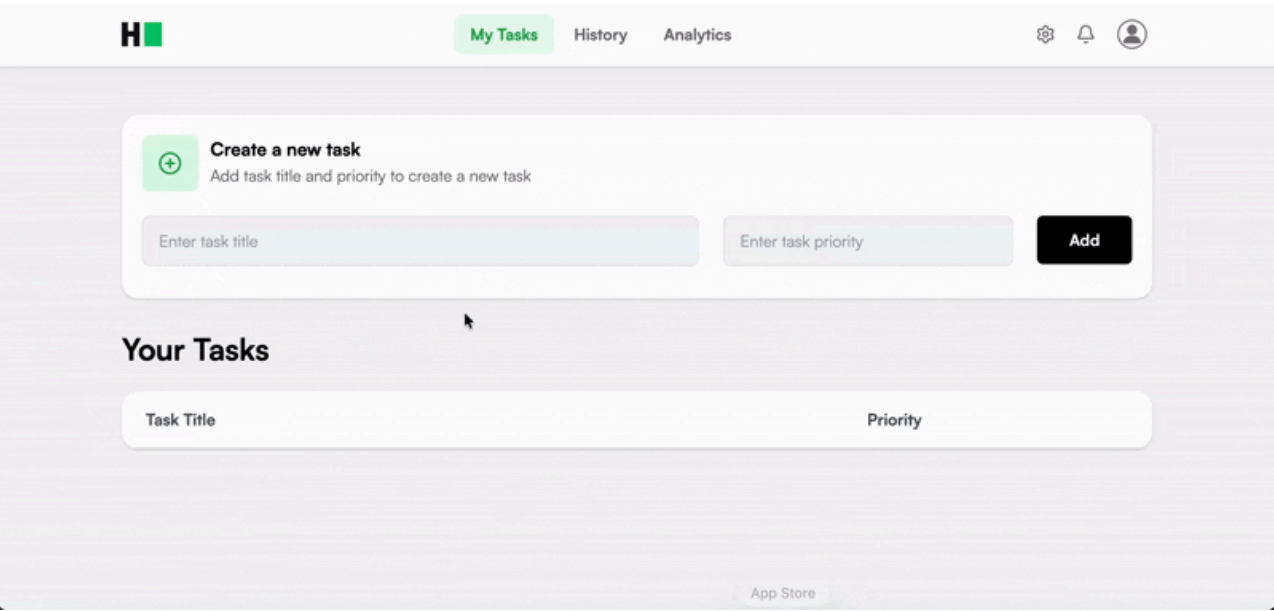
React: Task Manager

SCORE: 50 points

- React Hooks
- Easy
- React Component
- Front-End Development
- React States
- JavaScript

Implement a simple task manager where users can add, view, and delete tasks with priorities. All functionalities must be built inside the Tasks component.

[Hide animation](#) [Show animation](#)



Functional Requirements:

- Inputs
 - The Task Name input box must have type "text".
 - The Task Priority input box must have type "number".
- Initial State
 - The task table must be empty when the application starts.
- Adding a Task
 - Clicking the Add button should:
 - Add the task and its priority to the table.
 - Sort the tasks by priority (ascending order).
 - Reset both input boxes to empty.

- If either input box is empty, show an error toast notification with notificationan alert with the message: "Please enter both title and task priority" and do not add the task.

- Deleting a Task
 - Clicking the Delete button for a row should remove that task from the table.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
input-task-name	Input box for task name
input-task-priority	Input box for task priority
submit-button	Button for adding task
tasksList	List of tasks

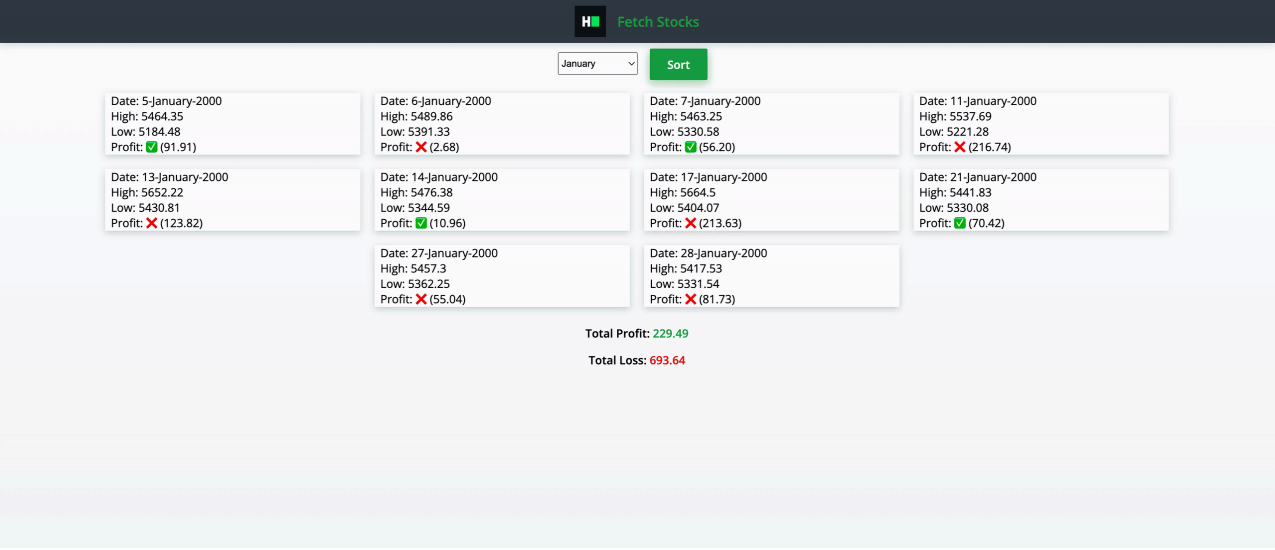
Question - 3
React: Fetch stocks

SCORE: 50 points

- React Hooks
- Easy
- React Component
- Front-End Development
- JavaScript

Complete the component as shown to pass the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The list of available stocks and their details is imported as `stocks` in the `Stocks` component. Use the list to render them as shown.

The data in the `stocks` file is in this form.

```
{
  "date": "5-January-2000",
  "open": 5265.09,
  "high": 5464.35,
  "low": 5184.48,
  "close": 5357
}
```

- The component must have the following functionalities:
- The dropdown menu should have the full names of all 12 months.
 - Initial UI:
 - "January" should be selected by default.
 - should display the stocks from "January" in unsorted form, and sums of profits and losses for the month.
 - Should display  in case of profit and x in case of loss in the Profit field along with the corresponding profit or loss in a stock, rounded to 2 decimal places.
 - Should display "Total Profit" and "Total Loss" for the selected month.
 - Selecting a month in the dropdown menu should display the details of that particular month in unsorted order.
 - Clicking the `Sort` button should sort the stocks shown from maximum profit to maximum loss.

Note: The profit is when the closing price of the stock is greater than the opening price and vice-versa for loss. For example, if "open" is 5265.09 and "close" is 5357, the profit is 5357 - 5265.09.

The following data-testid attributes are required in the component for the tests to pass.

Attribute	Component
select-month	Select menu for month
stocks	List of stocks for selected month
sort	Sort button
total-profit	Total Profit for selected month
total-loss	Total Loss for selected month

- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The file that should be modified by the candidate `src/components/Stocks/index.js` is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 4

React: Edit Profile

SCORE: 50 points

React

React Hooks

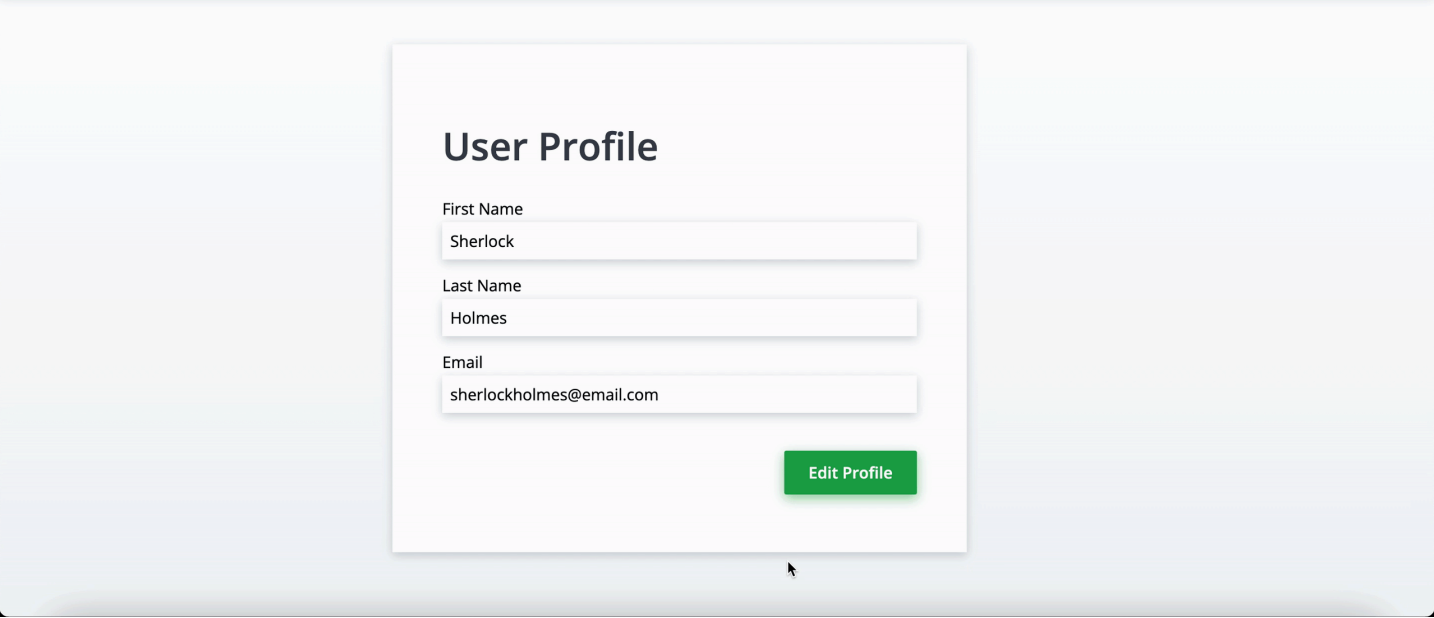
State Management

Easy

JavaScript

Complete a React Edit Profile application as shown in order to pass all the unit tests. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only 1 component:

- The Profile component enables the user to update his/her details.

The application has the following functionalities:

- The Profile component shows the user their details and allows editing.
- Initially, there should be 3 divs with the following values:
 - First Name: **Sherlock**
 - Last Name: **Holmes**
 - Email: **sherlockholmes@email.com**
- Initially, the button should have the text value Edit Profile.
- The following functionality should be implemented when the user clicks the 'Edit Profile' button.
 - The user can now edit the details in the divs.
 - The button should now have the text value "Save Changes".
- The following functionality should be implemented when the user edits the details and clicks the Save Changes button.
 - If one or more of the fields is empty, there should be an alert prompt with the text "Please enter all profile fields" and no values should be updated.
 - If all the fields are non-empty, there should be an alert prompt with the text "Profile updated successfully".
 - The input values should be the updated values and should be in uneditable divs.
 - The button text should be Edit Profile.

The following data-testid attributes are required in the component for the tests to pass:

Description	Name
First Name <div>	firstNameDiv
Last Name <div>	lastNameDiv
Email <div>	emailDiv
First Name <input>	firstNameInput
Last Name <input>	lastNameInput
Email <input>	emailInput
<button>.	changeButton

The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 5

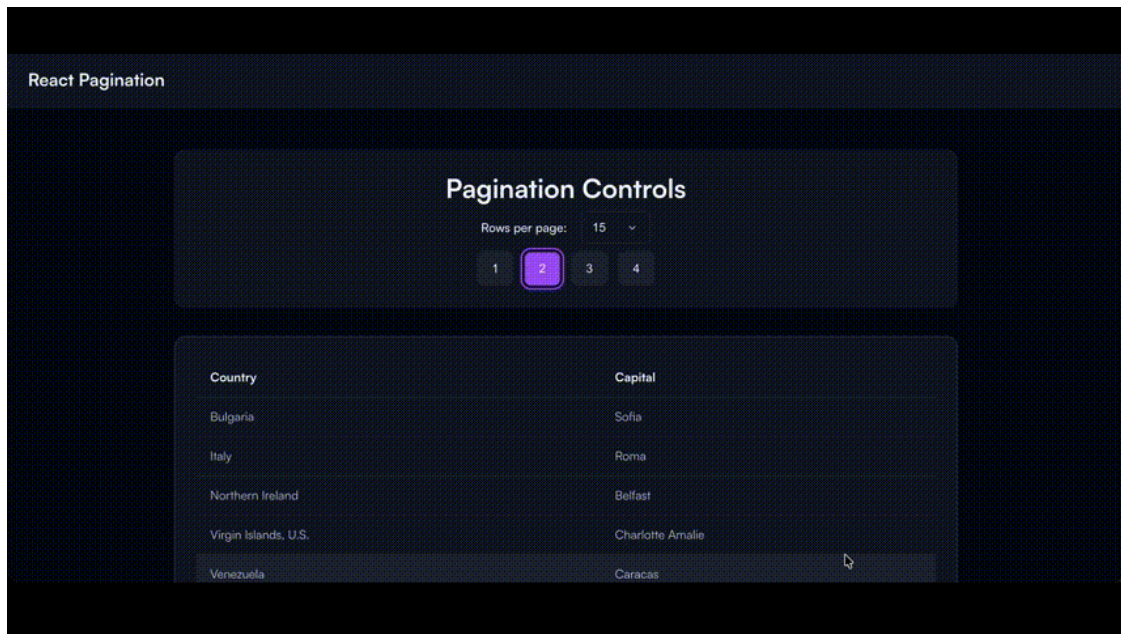
React: Pagination

SCORE: 50 points

React Easy React Component React Props React States JavaScript

Build a React application that displays a list of countries and their capitals in a paginated table. Users should be able to control how many rows appear per page and move between pages using pagination buttons. Some core React logic and data imports are already implemented for you.

[Hide animation](#) [Show animation](#)



Your users should be able to:

- View a table of countries and capitals using the imported data.
- Select how many rows are displayed per page from a dropdown with options **5**, **10**, and **15**.
- See 15 pagination buttons rendered on initial load.
- Change the number of rows displayed by selecting a different option, which should:
 - Update the number of rows shown in the table.
 - Recalculate and render only the necessary number of pagination buttons.
- Click any pagination button to view the corresponding subset of data.
- Always see table data that matches the order and content provided in `info.json`.

Note:

The following data-testid attributes are required in the component for the tests to pass:

- The 'Table' should have the data-testid attribute 'table'.
- The 'Table Body' should have the data-testid attribute 'tableBody'.
- The 'Buttons' header should have the data-testid attribute 'buttonDiv'.
- The 'Select' input should have the data-testid attribute 'selectInput'.

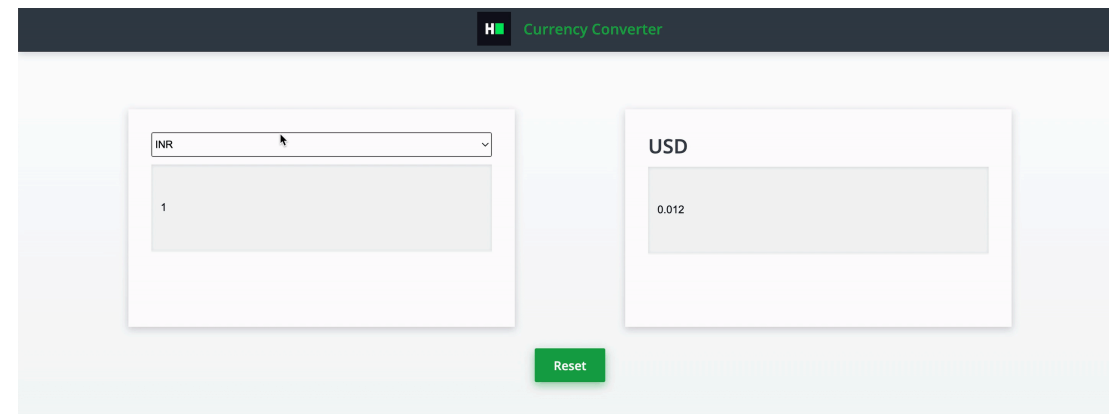
The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 1

React: Currency Converter

Complete the currency converter component that converts the value in any of the given currencies to USD as shown.

[Hide animation](#) [Show animation](#)



The list of available currencies is imported as "exchangeRates" in the component that can be used to render the list of currencies. The initial values for the currency and the input box are also imported from "exchangeRates.js".

It is given that:

- The first input box is to be used to take input.
- The second input box is read-only to display the converted value.
- Only positive integer or decimal values can be taken as input.

The component must have the following functionalities:

- The default value for:
 - input box should be 1.
 - the currency to display in the select menu should be INR.
 - The second input box should be the conversion of 1 INR to USD i.e. 0.012.
- The select menu should have five currencies in the options.
- The conversion should happen automatically:
 - when there is any input change.
 - on selecting another currency.
- The converted value should be up to 3 decimal places.
- The placeholder text should change automatically according to the currency.
- The Reset button should reset the input to 1 and give the conversion in the current currency.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
select menu	select-currency
first input box	input-value
second input box	converted-value

Clear Input button	clear-value
--------------------	-------------

Note:

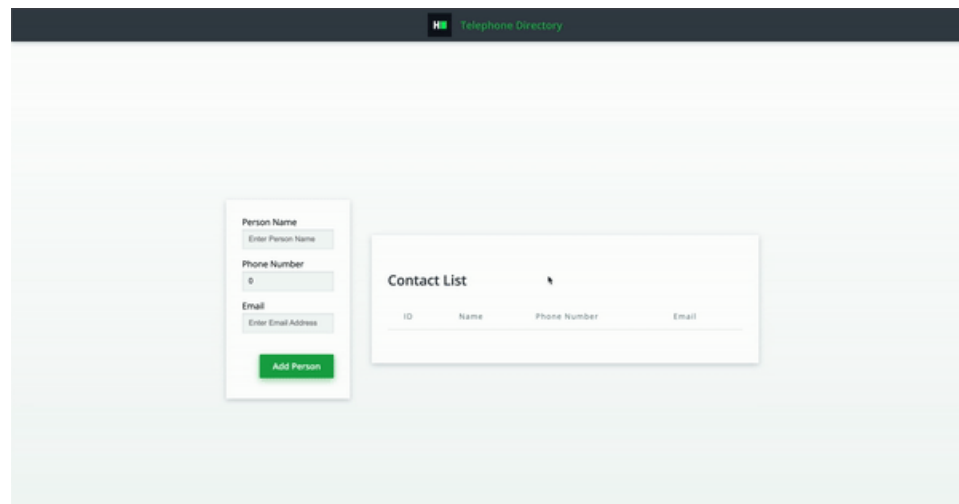
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is "src/components/currency-converter/index.js". It is open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 2

React (Debugging): Fix Telephone Directory

Debug the given Telephone Directory component to function as described below

[Hide animation](#) [Show animation](#)



The component must have the following functionalities:

- Rendering added contacts in real-time

1. AddPerson.js

- Name input should be of type **text**.
- The phone number should be of type **number** with a length of 10 digits.
- The email address should be of type **email**.
- No input field should be empty.
- On successful addition of a person, the person's data should be added to the list of people.

2. ListPeople.js

- The component should receive the contact list as an array of contact objects in props.

3. App.js

- The app should be responsible for communication between both components in the form of props.

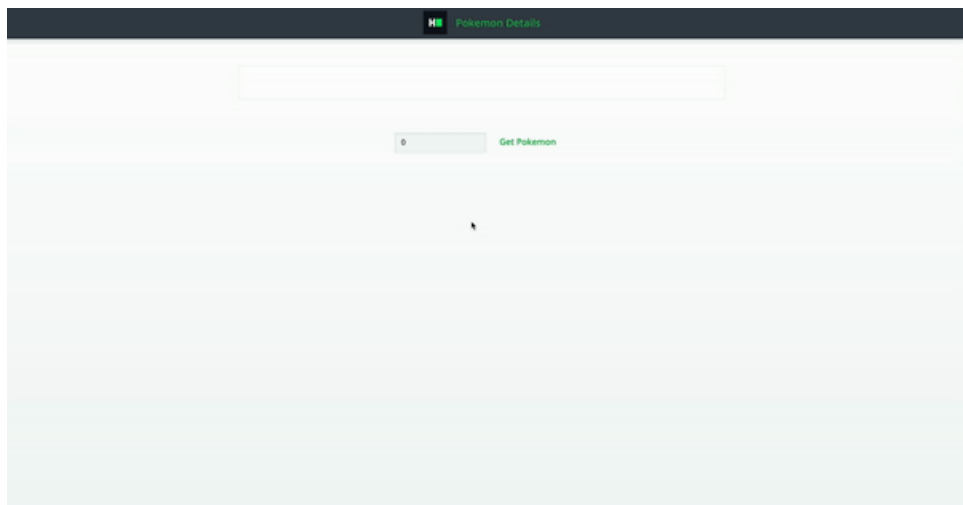
Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only files that should be modified are "/src/components/App.js", "src/components/AddPerson/AddPerson.js", and "src/components/ListPeople/ListPeople.js". They are open by default in the system editor.
- Candidates should avoid making any changes to other files.

Question - 3

Given a Pokemon details component, debug the React component to make it function as described.

[Hide animation](#)
[Show animation](#)



The component should have the following functionalities:

- The input field should take number input which is the pokemon id.
 - The id should be in range 1 and 151 inclusive.
 - Otherwise, the component should show an alert with the text 'Pokemon ID must be between 1 and 151'.
- On getting a valid pokemon id, the component should make a GET request to the API endpoint 'https://jsonmock.hackerrank.com/api/pokemon?id=<id>'. It returns pokemon data in JSON format.
 - e.g to get Pokemon with id = 1. the API endpoint will be: https://jsonmock.hackerrank.com/api/pokemon?id=1
 - For an API call with a valid pokemon ID, the response will be in this form:

```
{
  "data": {
    "id": 1,
    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ],
    "next_evolution": [
      {
        "num": "002",
        "name": "Ivysaur"
      },
      {
        "num": "003",
```

```
    "name": "Venusaur"
  }
}
}
```

- If the pokemon contains a next evolved version, the 'Next Evolution' button should be enabled.
- If the pokemon contains a previous evolved version(primitive version), the 'Prev Evloution' button should be enabled.
- On clicking the 'Next Evolution' button, the next evolved version of the pokemon should be fetched from the API and loaded.
- On clicking the 'Prev Evolution' button, the previous version of the pokemon should be fetched from the API and loaded.
- The Pokemon.js component should be used to render all the pokemon data by passing the pokemon as props.

Note:

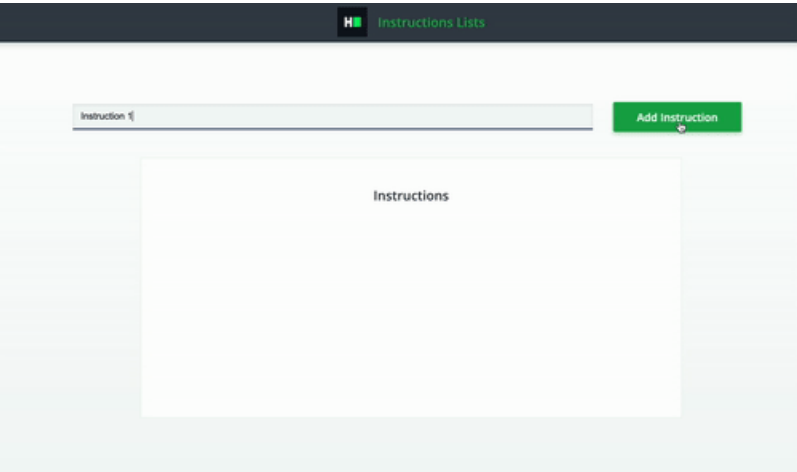
- Please note that components have the data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only files that should be modified are "/src/components/pokemon-details/PokemonDetails.js", and "src/components/pokemon/Pokemon.js". They are open by default in the editor.
- Candidates should avoid making any changes to other files.

Question - 4

React(Debugging):Fix Instructions Board

Given an instructions list component, debug the react component so it functions as described below.

[Hide animation](#) [Show animation](#)



Every instruction should be rendered as an element of the unordered list (ul).

The list and all the instructions in it should satisfy the following requirements:

- Each instruction should be wrapped inside an .
- Each instruction should have the 'Sr. No.' rendered along with it.
- The instructions should have 2 buttons: Move Up and Move Down.
 - Move Up: On clicking this button, the instruction should be moved up one position on the list.
 - It should be disabled for the topmost instruction.
 - Move Down: On clicking this button, the instruction should be moved down on the list.
 - It should be disabled for the bottommost instruction.
- On changing the order of instructions, the 'Sr No' rendered with it should change.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified is "/src/components/instructions-board/index.js". It is open by default in the editor.
- Candidates should avoid making any changes to the rest of the files in the project structure.

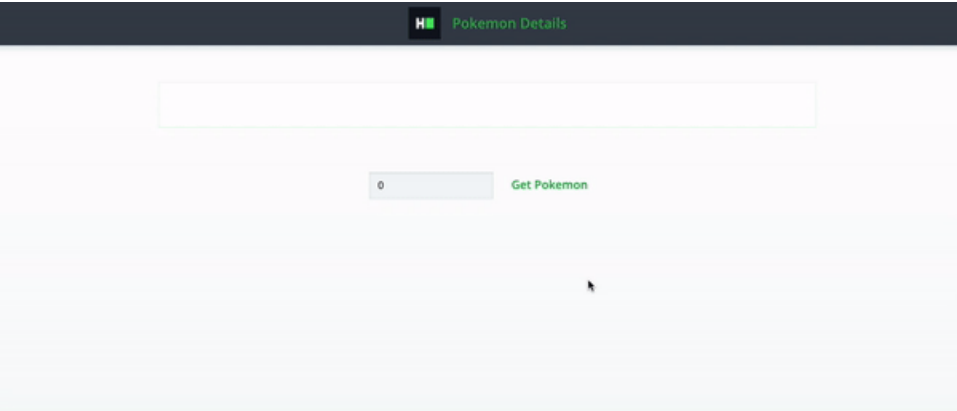
Question - 5

React: Pokemon Details

After twenty-one years, Ash Ketchum was finally a Pokémon Champion. He decided to retire from Pokemon training and started software development. He is creating a pokemon search application, which takes a pokemon id as input and searches the pokemon database for the pokemon and its evolved and primitive versions.

Help Ash to complete the React Components as shown below.

[Hide animation](#) [Show animation](#)



The component should have the following functionalities:

- The input field should take the pokemon id as a number.
- The id should be in the range of 1 to 151. Otherwise, the component should show an alert with the text 'Pokemon ID must be between 1 and 151'.
- On getting a valid pokemon id, the component should make a GET request to the following API endpoint 'https://jsonmock.hackerrank.com/api/pokemon?id=<id>' which returns pokemon data in JSON format.
 - e.g. to get Pokemon with id = 1, the API endpoint will be https://jsonmock.hackerrank.com/api/pokemon?id=1
 - For an API call with a valid pokemon ID, the response will be:

```
{
  "data": {
    "id": 1,
    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ]
  },
}
```

```
"next_evolution": [
  {
    "num": "002",
    "name": "Ivysaur"
  },
  {
    "num": "003",
    "name": "Venusaur"
  }
]
}
```

- If the pokemon contains the next evolved version, the 'Next Evolution' button should be enabled.
- If the pokemon contains a previous evolved version (primitive version), the 'Prev Evolution' button should be enabled.
- On clicking the 'Next Evolution' button, the next evolved version of the pokemon should be fetched from the API and loaded.
- On clicking the 'Prev Evolution' button, the previous version of the pokemon should be fetched from the API and loaded.
- The Pokemon.js component should be used to render all the pokemon data by passing the pokemon as props.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is `"/src/components/pokemon-details/PokemonDetails.js"`, `"src/components/pokemon/Pokemon.js"` and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 1

SCORE: 50 points

React: Movie Booking

Front-End Development

React

Easy

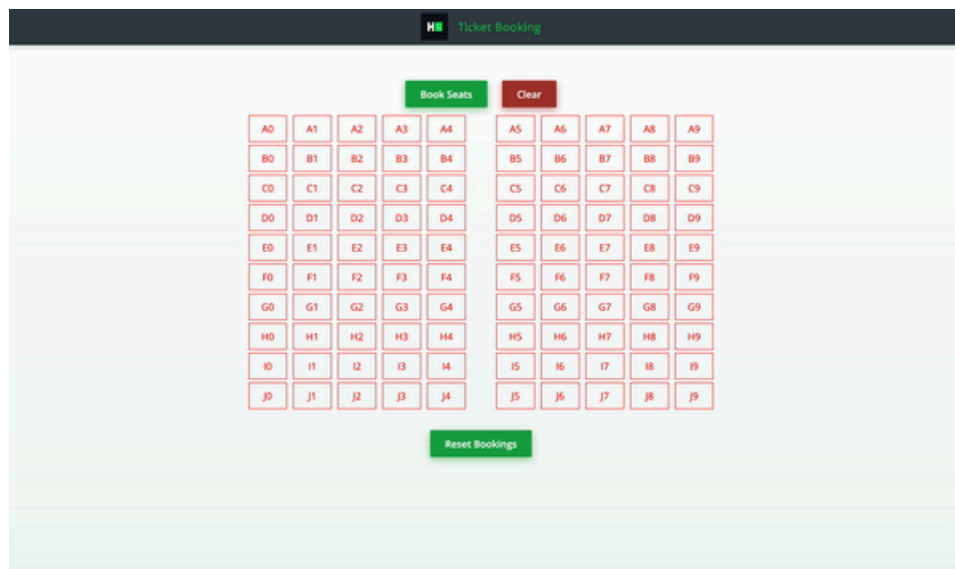
Component State Management

Functional Components

Hooks

JavaScript

There is a new cinema hall opening in HackerWorld and the owner wants an application to book tickets online. Complete the booking application as described.

[Hide animation](#) [Show animation](#)

The seats in the cinema hall are represented as a diagram. A user should be able to select seats and book them. The owner will have the Reset Bookings option which will reset all bookings.

The component must have the following functionalities:

- Initially, the className for every seat is 'seat'.
- When a user clicks on an unselected seat, it is selected and its className should be 'selected seat'.
- When a user clicks on a selected seat, it is deselected and its class name should revert to 'seat'.
- When a user clicks on the 'Book Seats' button:
 - If no seats are selected, it should alert 'Please select at least one seat'.
 - Selected seats should be added to a list of booked seats and their class should be 'disabled seat'.
- On clicking the 'Clear' button, all seats that are not booked should be unselected.
- On clicking the 'Reset Bookings' button, the application should go back to the initial state. i.e. All the selected and booked seats should get reset and the className for all seats should be 'seat'.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "src/components/TicketBooking.js" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 2

SCORE: 50 points

React: Team Selection

React

Easy

Component State Management

Functional Components

Hooks

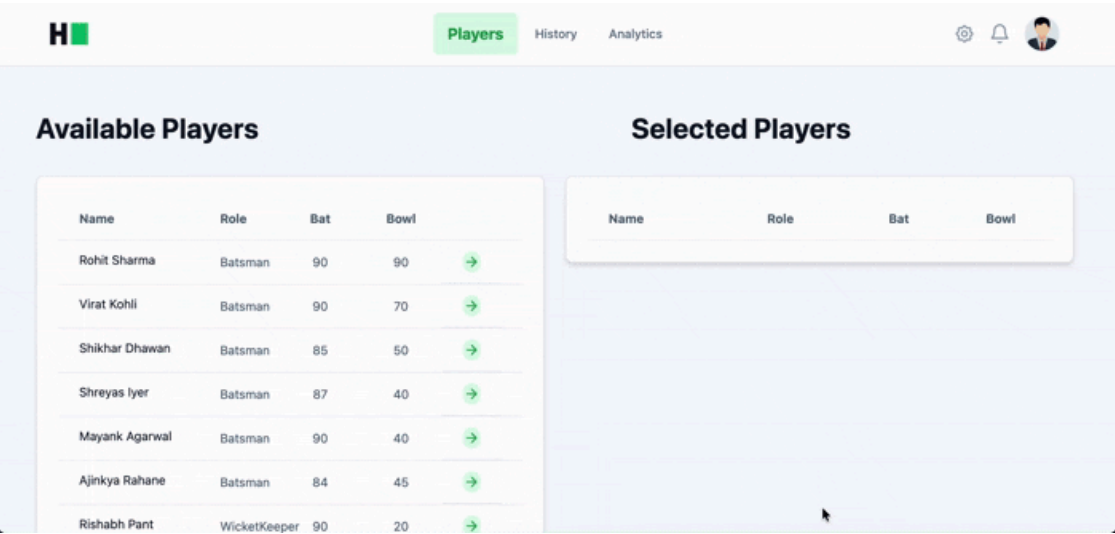
Front-End Frameworks

JavaScript

The Indian Cricket Team coach is selecting the team for the World Cup. Create a component using ReactJS that will display a list of available players, their roles in the team along with their batting and bowling skills attributes. It should be able to sort the list by any attribute, i.e. name, role, batting, and bowling skills.

Complete the Team Selection component as shown below.

[Hide animation](#) [Show animation](#)



The list of available players is imported as 'playersList' in the component. Use the list to render the Available Players as shown in the app.

The component must have the following functionalities.

- Each player in playersList has the following attributes:
 - Name: The name of the player
 - Type: The role of the player in the team (Batting, Bowling, AllRounder and WicketKeeper)
 - BattingSkill: It defines the Batting Skill of the player
 - BowlingSkill: It defines the Bowling Skill of the player
 - FieldingSkill: It defines the Fielding Skill of the player
- The players' name, role, batting, and bowling skills should be rendered as-is.
- In the available players list:
 - There should be a Select button for every player in the row.
 - On clicking the Select button, the button should be disabled and the player must be moved to the selected players list.
 - On clicking Name in the table header, sort the players list by name, ascending.
 - On clicking Role in the table header, sort the players list by role, ascending.
 - On clicking Bat in the table header, sort the players list by BattingSkill, ascending.
 - On clicking Bowl in the table header, sort the players list by BowlingSkill, ascending.
- In the selected players list, on clicking the Remove button:
 - the player must be removed from the selected players list.
 - the player's Select button must be enabled in the available players list.
- There is no requirement of sorting players in any order for Selected players

The following data-tested attributes are required in the component for the tests to pass:

- For every <tr> in 'Available Players' table, it must contain data-testid: 'available-<playerfirstname>-<player lastname>-row'.
 - e.g for Rohit Sharma, the data-testid will be 'available-Rohit-Sharma-row'
- For every <td> in <tr>, it must contain data-testid: 'available-<playerfirstname>-<playerlastname>-attribute'.
 - e.g for <td> with name Rohit Sharma, the data-testid will be 'available-Rohit-Sharma-name'
 - e.g for <td> with type for Rohit Sharma, the data-testid will be 'available-Rohit-Sharma-type'
- For every <button> (Player Select Button), it must contain data-testid: 'available-<playerfirstname>-<playerlastname>-select'.
 - e.g for <button> to select Rohit Sharma, the data-testid will be 'available-Rohit-Sharma-select'

```
<tr data-testid="available-Rohit-Sharma-row">
  <td data-testid="available-Rohit-Sharma-name">Rohit Sharma</td>
  <td data-testid="available-Rohit-Sharma-role">Batsman</td>
  <td data-testid="available-Rohit-Sharma-bat">90</td>
  <td data-testid="available-Rohit-Sharma-bowl">90</td>
  <td>
```

```
<button
  data-testid="available-Rohit-Sharma-select"
  class="btn-primary text"
>
  Select
</button>
</td>
</tr>
```

- For every <tr> in 'Selected Players' table, it must contain data-testid: 'available-<playerfirstname>-<player lastname>-row'.
 - e.g for Rohit Sharma, the data-testid will be 'selected-Rohit-Sharma-row'
- For every <button> in Selected Players Row (Player Remove Button), it must contain data-testid: 'selected-<playerfirstname>-<playerlastname>-remove'
 - e.g for <button> to select Rohit Sharma, the data-testid will be 'selected-Rohit-Sharma-remove'

```
<tr data-testid="selected-Rohit-Sharma-row">
  <td>Rohit  Sharma</td>
  <td>Batsman</td>
  <td>
    <button data-testid="selected-Rohit-Sharma-remove" class="btn danger text">
      Remove
    </button>
  </td>
</tr>
```

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "/src/components/team-selection/index.js" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 3

React: Editable Table

SCORE: 50 points

- Easy
- React
- Component State Management
- Cross Platform
- Front-End Frameworks
- Hooks
- JavaScript

There is a partial React editable table application. Certain core React functionalities have already been implemented. Complete the application as shown below in order to pass all the unit tests.

[Hide animation](#) [Show animation](#)

Name	Position	Salary	Action
Chris Hatch	Software Developer	130000	Save
Elizabeth Montgomery	Lead Research Engineer	70000	Save
Alden Shaw	Machine Learning Engineer	80000	Save
Enter Name	Enter Position	Enter Salary 	Add

The application has 3 components:

- The App component, which renders the table with employee data.
- The Employee component, which forms an employee entry with name, position, and salary.
- The AddEmployee component, which consists of multiple inputs for adding a new employee entry.

The application has the following functionalities:

- For the Employee component:
 - Each employee object has the following keys:
 - id: The unique ID of the employee (Integer)
 - name: The name of the employee (String)
 - position: The current role of the employee in the company (String)
 - salary: The current salary of the employee (Integer)
 - Initially, the "Save" buttons for saving a new salary are disabled.
 - The salary field becomes editable when clicked.
 - The "Save" button for a specific employee is only enabled when the edited salary value is set and is different from the existing value.
 - Clicking the "Save" button updates the salary field with the new value, and the salary field again becomes not editable.
- For the AddEmployee component:
 - Each new employee added to the list should have a unique incremental ID property attached to it.
 - Initially, the "Add" button for adding a new employee entry is disabled.
 - The "Add" button becomes enabled only when all fields are filled.
 - Clicking the "Add" button adds a new employee entry to the table and resets input fields.

The following data-testid attributes are required in the component for the tests to pass:

- The table should have the data-testid attribute 'table'.
- The employee uneditable salary fields should have the data-testid attributes 'employee-salary-div-0', 'employee-salary-div-1', and so on.
- The employee salary inputs should have the data-testid attributes 'employee-salary-input-0', 'employee-salary-input-1', and so on.
- The "Save" buttons should have the data-testid attributes 'employee-save-button-0', 'employee-save-button-1', and so on.
- The new employee name input should have the data-testid attribute 'new-employee-name-input'.
- The new employee position input should have the data-testid attribute 'new-employee-position-input'.
- The new employee salary input should have the data-testid attribute 'new-employee-salary-input'.
- The "Add" button should have the data-testid attribute 'add-new-employee-button'.

Please note that the component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 4

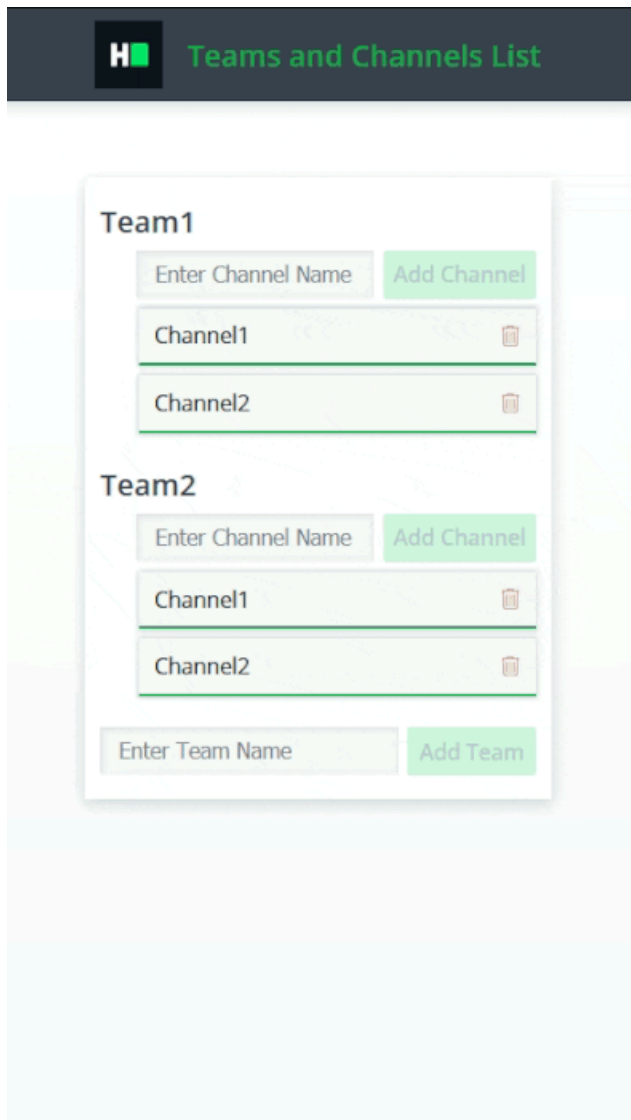
SCORE: 50 points

React: Teams and Channel List Application

React Front-End Frameworks Easy Component State Management Hooks Cross Platform JavaScript

There is a partially completed React teams and channel list application. Certain core React functionalities have already been implemented. Complete the application as shown below in order to pass all the unit tests.

[Hide animation](#) [Show animation](#)



The application has 2 components:

- The TeamList component, which presents a list of teams and an input to add a new team.
- The Team component, which displays a list of channels for each team and an input to add a new channel.

The application has the following functionalities:

- For the TeamList component:
 - Each team in the teams array has the following keys:
 - name: The name of the team. (String)
 - channels: The list of channels belonging to a team. (Channel Array)
 - Initially, the "Add Team" button is disabled.
 - It becomes enabled only when a **unique**, valid team name having at least one character is entered in the input.
 - Upon clicking the enabled "Add Team" button, the team is added to the Teams List.
- For the Team component:
 - Each channel in a team has the following keys:
 - name: The name of the channel. (String)
 - id: The unique ID for the channel. (Integer)
 - Initially, the "Add Channel" button is disabled.
 - It becomes enabled only when a **unique**, valid channel name having at least one character is entered in the input.
 - Upon clicking the enabled "Add Channel" button, the channel is added to the Team component.
 - While adding a new channel, a new unique incremental ID property must be provided.
 - Clicking the *delete* button on any channel removes it from the list.

The following data-testid attributes are required in the component for the tests to pass:

- The div with the list of teams should have the data-testid attribute 'team-list'.
- The lists of channels for each team should have the data-testid attributes 'channel-list-0', 'channel-list-1', and so on.
- The 'Add Team' button should have the data-testid attribute 'add-team-btn'.
- Each 'Add Channel' button should have the data-testid attribute 'add-channel-btn-0', 'add-channel-btn-1', and so on.
- The 'Enter Team Name' input should have the data-testid attribute 'team-name-input'.
- Each 'Enter Channel Name' input should have the data-testid attribute 'channel-name-input-0', 'channel-name-input-1', and so on.
- Each *delete* button should have the data-testid attribute 'remove-channel-button-00', 'remove-channel-button-01', and so on, denoting first the team id and then the channel id.

Please note that component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 5

React: Caesar's Cipher

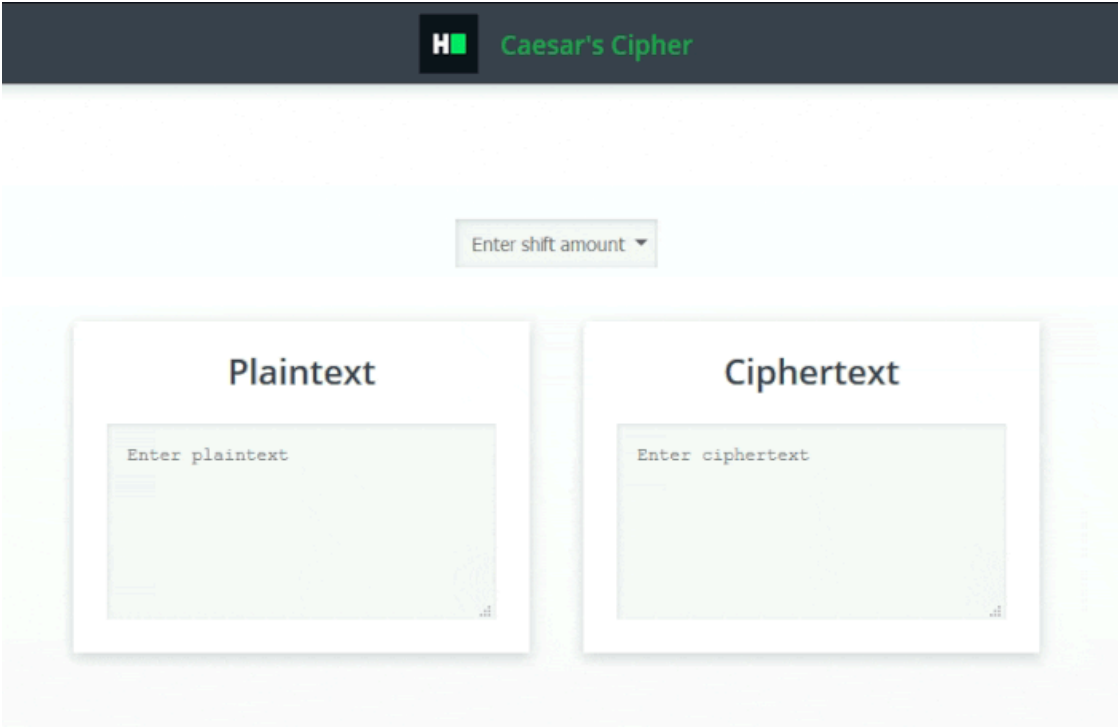
SCORE: 50 points

- React
- Front-End Frameworks
- Easy
- State Management
- Hooks
- JavaScript

Complete a partially completed React application that encrypts and decrypts a piece of text using [Caesar's Cipher](#). The cipher uses an encryption algorithm where each letter is replaced with a letter by some fixed number of positions down the alphabetical series (e.g., a shift of 1 replaces the letter *a* with *b*, the letter *e* with *f*, the word *hello* with *ifmmp*, etc.).

Certain core React functionalities have already been implemented. Complete the React application as shown below in order to pass all the unit tests.

[Hide animation](#) [Show animation](#)



- The application has the following functionalities:
- The app has three components:
 - The Shift component, which allows the user to enter the number of places to shift each character.
 - The Plaintext component, which allows the user to enter plain text.
 - The Ciphertext component, which allows the user to enter encrypted text.
 - The shift component initially shows the text 'Enter shift amount'.
 - A string is encrypted when the user does the following:

- Selects the number of units to shift from the dropdown.
- Enters the text to encrypt into the *Plaintext* textbox.
- A string is decrypted when the user does the following:
 - Selects the number of units to shift from the dropdown.
 - Enters the text to decrypt into the *Ciphertext* textbox.
- The following list of actions should be supported by the app:
 - Enter the text to encrypt into the *Plaintext* textbox, which encrypts and renders correctly in the *Ciphertext* textbox
 - Delete text from either textbox
 - Enter the text to decrypt into the *Ciphertext* textbox, which decrypts and renders correctly in the *Plaintext* textbox
- The cipher is case-sensitive.
- During conversion, you may ignore numbers and special characters with the exception of spaces.

The following data-testid attributes are required in the component for the tests to pass:

- The select field to choose the shift should have the data-testid attribute 'select'.
- The *Plaintext* textbox should have the data-testid attribute 'plain-text'.
- The *Ciphertext* textbox should have the data-testid attribute 'cipher-text'.

Please note that the component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Note: The functions for encrypting and decrypting the data are provided in the Ciphertext and Plaintext components respectively. Please use these functions to encrypt/decrypt the text.

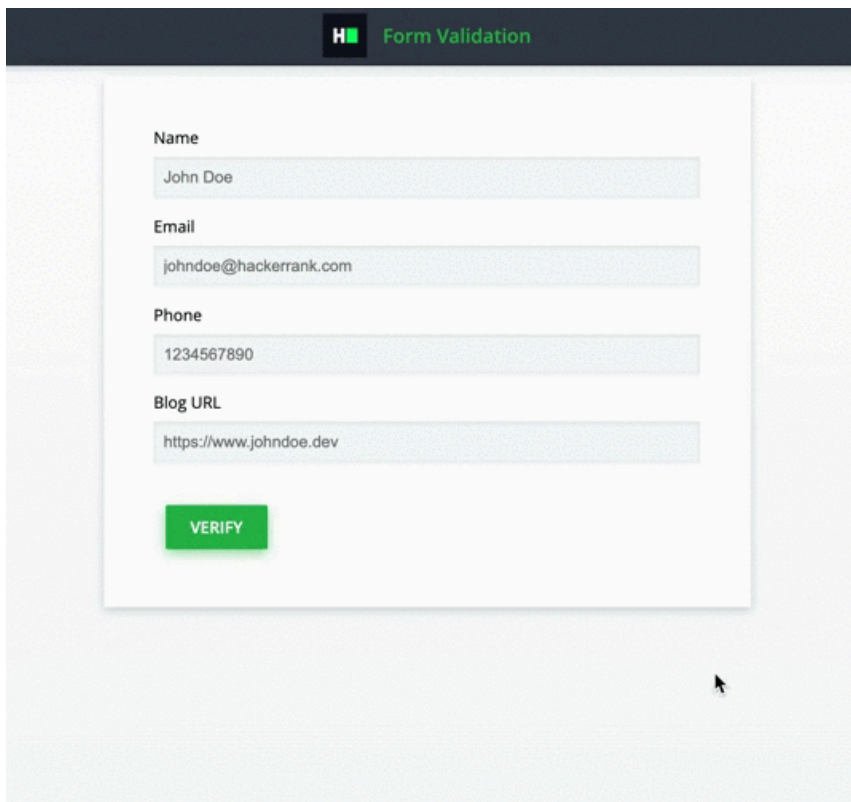
Question - 1

Form Validation

Given a partially completed React application with the HTML template and few built-in functionalities, your task is to validate the fields in the form based on few constraints that are specified.

Certain core React functionalities have already been implemented. Complete the React application as shown below in order to pass all the tests.

[Hide animation](#) [Show animation](#)



The screenshot shows a web application with a dark header bar containing the 'H' logo and the text 'Form Validation'. Below the header is a light gray form container. Inside the container, there are four input fields, each with a label above it: 'Name' (containing 'John Doe'), 'Email' (containing 'johndoe@hackerrank.com'), 'Phone' (containing '1234567890'), and 'Blog URL' (containing 'https://www.johndoe.dev'). At the bottom of the form container is a green button with the text 'VERIFY' in white. A mouse cursor is visible near the bottom right of the form container.

The application has one *Form Component* that holds 4 input fields 'Name', 'Email', 'Phone' and 'Blog Url'

The app should implement the following functionalities:

- The user should be able to fill in the contents of all 4 fields.
- When verify button is clicked, the values in all the fields should be validated as per the constraints.
- The Error message fields should be rendered below the corresponding field only if the validation for that field fails.
- If the values in all the fields are valid, on clicking the `verify` button should show the `Verification Successful` message below it.

The validation rules for each fields are given below:

- The `Name` field should accept only alphabets and empty spaces. The name should contain at least 3 alphabets. The total length of the name should not exceed 25 characters. If the Name is invalid, it should show *Error: Invalid Name* as error message.
- The `Email` field use `validateEmail` function from `utils.js` to validate the email. If the email is invalid, it should show *Error: Invalid Email Address* as error message.

- The `Phone` field should accept only numbers and its length should be exactly 10.If the phone number is invalid, it should show `Error: Invalid Phone Number` as error message.
- The `Blog URL` field should use `validateUrl` function from `utils.js` to validate the Blog's URL.If the URL is invalid, it should show `Error: Invalid URL` as error message.

Note: The `utils.js` file holds the validation logic for `email` and `url` fields through `validateEmail` and `validateUrl` functions.Both of them accepts a string and returns a boolean that indicates if it is valid or not.

The following data-testid attributes are required in the component for the tests to pass:

- The `Name` <input> tag should have the data-testid `name` and its corresponding element to show error message should have `nameErrorTxt` as its data-testid.
- The `Email` <input> tag should have the data-testid `email` and its corresponding element to show error message should have `emailErrorTxt` as its data-testid.
- The `Phone` <input> tag should have the data-testid `phone` and its corresponding element to show error message should have `phoneErrorTxt` as its data-testid.
- The `Blog URL` <input> tag should have the data-testid `url` and its corresponding element to show error message should have `urlErrorTxt` as its data-testid.
- The Verify button should have the data-testid `verify`.
- The element that shows the success message should have the data-testid `successMsg`.

Please note that the component has the above data-testid attributes for test cases. It is advised not to change them.

Question - 2

React: Birthday Records

Complete a Birthday Records component that sorts a list of people by name or by age.

Certain core React functionalities have already been implemented. Complete the React application as shown below in order to pass all the unit tests.

[Hide animation](#) [Show animation](#)

Sort By ☒ Name ☐ Age

Person Name	Date of Birth
Rhianna Johnson	11/30/2011
Aiden Shaw	09/16/1992
Trevon Floyd	01/16/1992
Melanie Yates	12/12/2001
Chris Andrews	02/09/1891
Jacinda Miller	12/01/1982
Will Davis	11/30/2011
Eliza Baxter	10/31/1999

The component has the following functionalities:

- The application's initial state displays a table where the unsorted list of people records must be rendered.
- The radio buttons to sort by 'Name' or 'Age' are toggled when clicked, so that only one is selected at a time.
- Clicking the radio button to sort by 'Name' must sort the list by *ascending name*.
- Clicking the radio button to sort by 'Age' must sort the list by *descending age* (i.e., *ascending date*).

The following data-testid attributes are required in the component for the tests to pass:

- The radio button to sort by 'Name' should have the data-testid attribute 'name'.
- The radio button to sort by 'Age' should have the data-testid attribute 'age'.
- The table displaying the names and dates should have the data-testid attribute 'table'.

Please note that component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 3

React: Country Filter

Given a partially completed React application with the HTML template built and ready, implement a filter that searches and displays matching countries in a list.

[Hide animation](#) [Show animation](#)



Enter Country Name

Afghanistan

Albania

Algeria

Andorra

Angola

Anguilla

Antigua & Barbuda

Argentina

Armenia

The application has 2 components:

1. The `Search` view, which has the input box where the user can type to filter countries.
2. The `CountryList` view, which renders the list of countries that are filtered.

The entire list of countries to display is stored in a variable named `response` inside the file `src/response.js`.

The app should implement the following functionalities:

- The initial view should render an empty input box with all countries appearing in the same order as in the `response` variable.
- When a character is typed, each country containing the set of typed characters should be filtered and displayed.
- The filtering should be case insensitive.
- When there is no character typed in the search box, it should show all countries.

Note:

- Get the list of countries to be displayed from the `response.js` file.
- The order in which each country appears should follow the same order in the `response.js` file, even after filtering.

The following data-testid attributes are required in the component for the tests to pass:

- The country filter field `<input>` tag should have the data-testid attribute 'filterInput'.
- The countries wrapper `<ul>` tag should have the data-testid attribute 'countryList'.

Please note that the component has these data-testid attributes for test cases. They should not be changed.

Question - 4

React: Autocorrection App

Create a basic autocorrection application per the requirements below. The finished application must pass all of the unit tests.

[Hide animation](#) [Show animation](#)

I

Complete the implementation of `src/components/AutocorrectTextarea.js` according to the following requirements:

- AutocorrectTextarea is a component that takes a corrections Object that maps strings to their corrections. For example, the object below denotes that 'really' is a correction for 'realy', and 'weird' is a correction of 'wierd':

```
const corrections = {  
  'realy': 'really',  
  'wierd': 'weird',  
};
```

- Assume that no value of the corrections object appears as the property in the corrections object.
- AutocorrectTextarea renders a textarea element and lets users write text in it.
- Assume that the text consists only of words separated by a single space character.
- Once a space character is typed, the word preceding it is considered to be complete and must be autocorrected according to the corrections object if a correction exists.

Initially, the file is filled with boilerplate code. Note the following:

- The textarea element must have `data-testid="textarea"`.

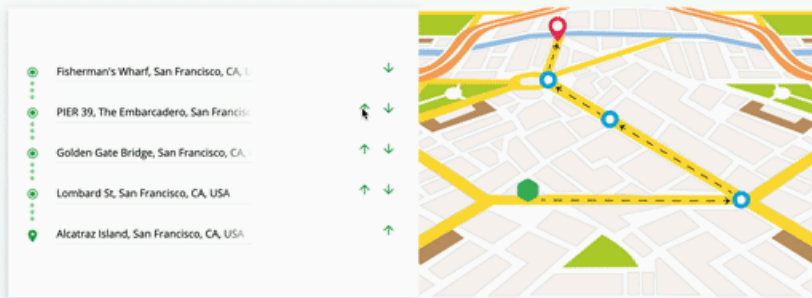
Please note that the component has these data-testid attributes for test cases, and certain classes and ids for rendering purposes. You should not change them.

Question - 5

React: HackerMaps

Implement a basic navigation application, as shown below. Some core functionalities have already been implemented, but the application is incomplete. Requirements are given below the image, and the finished application must pass the unit tests.

[Hide animation](#) [Show animation](#)



Requirements:

1. The app consists of one component: the Navigation view
2. The location list is provided as props to the Navigation component
3. Locations should be displayed as `<li>` tags in the same order they are provided
4. Each location can have one or two icon buttons:
 - First location: Only the Move Down button
 - Last location: Only the Move Up button
 - All other locations: Both Move Up and Move Down buttons
5. Button functionality:
 - Clicking Move Down: Moves the location down one position
 - Clicking Move Up: Moves the location up one position
 - When a location moves, it exchanges positions with the adjacent location

Note: The utility functions `isLast` and `getClasses` are provided in the template. Do not modify `getClasses`.

Required `data-testid` attributes:

- 'location-list' for the parent container `<ul>`
- 'location-0', 'location-1', etc. for each location item
- 'location' for each location name `<p>` tag
- 'up-button' for each Move Up button
- 'down-button' for each Move Down button

Do not change the `data-testid` attributes, classes, or IDs as they are required for testing and rendering.

Question - 1

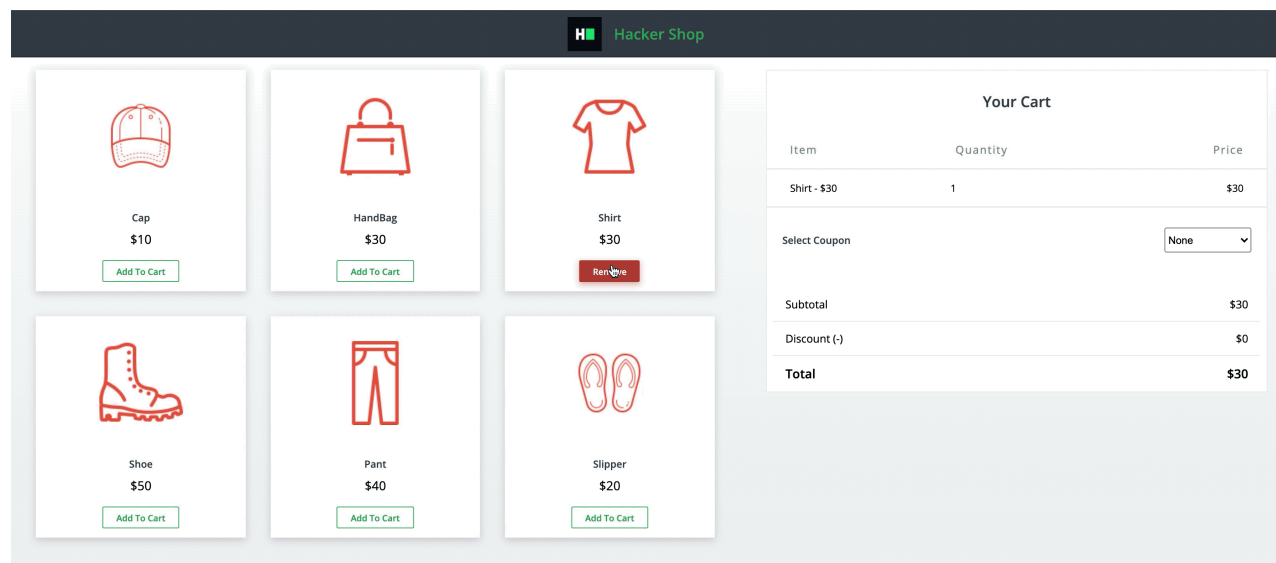
React: HackerShop Checkout

SCORE: 50 points

React Easy Component State Management Theme: E-commerce Class Components JavaScript

Create a basic shopping application, as shown below. Some core functionalities have already been implemented, but the application is not complete. Application requirements are given below, and the finished application must pass all of the unit tests.

[Hide animation](#) [Show animation](#)



The app has two separate views/components: the Product Listing component and the Cart component. The list of products to be displayed is already provided in the app.

The app should implement the following functionalities:

- Clicking on each 'Add To Cart' button should add the item to the shopping cart. When an item is added to the cart:
 - The 'Add To Cart' button should be removed from view, and the 'Remove From Cart' button should be displayed.
 - An entry should be added to the table in the Cart component.
- Clicking on each 'Remove' button should remove the item from the cart and display 'Add to Cart' for the product item.
- The Cart component should have the following functionalities:
 - Display all the items in the cart in a table.
 - Display the cart's subtotal, discount value, and total price.
 - The cart has a 'Select Coupon' input. On selecting a coupon from this input, an appropriate discount is applied and the total price is calculated and displayed. (Subtotal - Discount = Total Price)
- Items should be displayed in the Cart component in the order they are added to the cart.
- The list of products and the cart object are passed as props to the Product Listing component and the Cart component respectively.

Each product object contains the following properties:

- name: Name of the product. [STRING]
- price: Price of the product. [NUMBER]
- id: Unique ID of the product. (Auto Generated) [NUMBER]
- image: The image URL of the product. [STRING]
- cartQuantity: The quantity of the item in the cart. The default value should be 0. [NUMBER]

- Each item in the cart, CartItem, has the following properties:
- id: The ID of the product added to the cart. [NUMBER]
 - item: The heading property of the product. [STRING]
 - quantity: The quantity of the item in the cart. [NUMBER]
 - price: The total price of the item in the cart. (quantity x product.price) [NUMBER]

- The following data-testid/class attributes are required in the component for the tests to pass:
- Each product item in the Product Listing component should have the class 'product-item'.
 - Each 'Add to Cart' button should have the data-testid attribute 'btn-item-add'.
 - Each 'Remove' button should have the data-testid attribute 'btn-item-remove'.
 - The table rows <tr> in the Cart component, corresponding to items in the cart, should have the data-testid attribute of 'cart-item-0', 'cart-item-1', and so on.
 - The table data <td>, containing the name of the item in the cart, should have the data-testid attribute 'cart-item-name'.
 - The table data <td>, containing the quantity of the item in the cart, should have the data-testid attribute 'cart-item-quantity'.
 - The table data <td>, containing the price of the item in the cart, should have the data-testid attribute 'cart-item-price'.
 - The 'Select Coupon' input should have the data-testid attribute 'cart-coupon'
 - The cart's 'Subtotal' value container should have the data-testid 'cart-subtotal'.
 - The cart's 'Discount' value container should have the data-testid 'cart-discount'.
 - The cart's 'Total Price' value container should have the data-testid 'cart-total'.

Please note that the component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 2

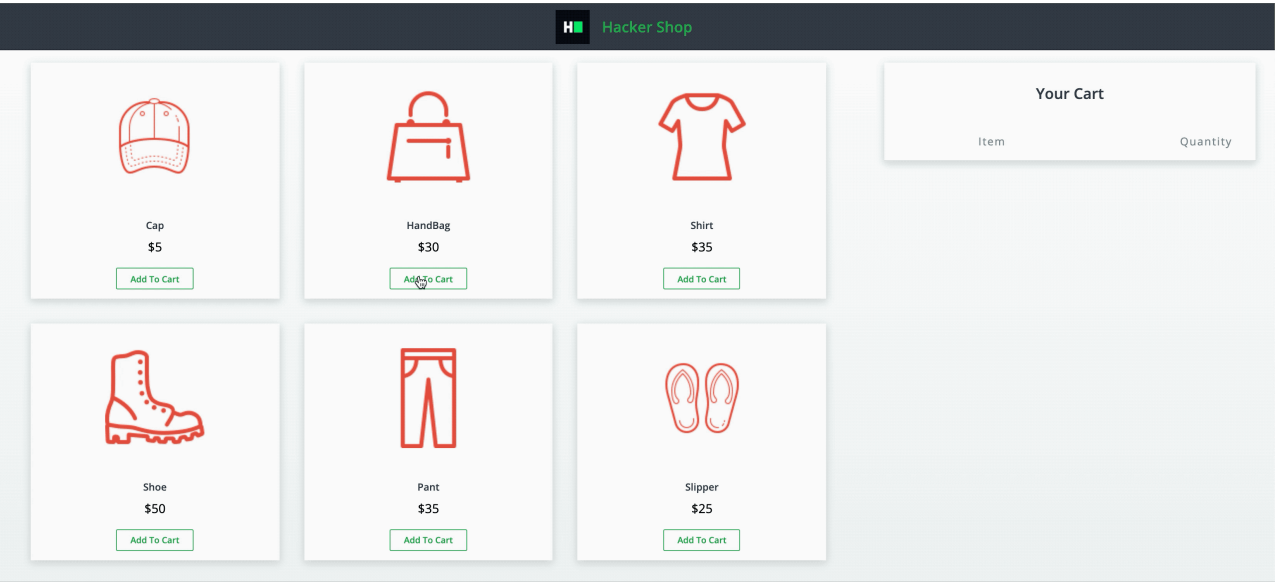
React: HackerShop Shopping Cart

SCORE: 50 points

- React
- Component State Management
- Easy
- Theme: E-commerce
- Class Components
- JavaScript

Create a basic shopping application, as shown below. Some core functionalities have already been implemented, but the application is not complete. Application requirements are given below, and the finished application must pass all of the unit tests.

[Hide animation](#) [Show animation](#)



The app has two separate views/components: the Product Listing component and the Cart component. The list of products to be displayed is already provided in the app.

The app should implement the following functionalities:

- Clicking on each 'Add To Cart' button should add the item to the shopping cart. The listing in the Product Listing component should be updated to show the 'Increase/Decrease Quantity' button and the quantity of the item in the cart.
- Clicking on each 'Increase Quantity' button should increase the quantity of the item in the cart.
- Clicking on each 'Decrease Quantity' button should do the following:
 - When the cart quantity of the item is 1: This should remove the item from the cart, hide the 'Increase/Decrease Quantity' button, and show the 'Add to Cart' button.
 - When the cart quantity is greater than 1: The quantity of the item in the cart should be decreased.
- On every quantity update operation, the text for the quantity of the item should be updated both in the Product Listing component as well as in the corresponding entry in the Cart component.
- Items should be displayed in the Cart component in the order they are added to the cart.
- The list of products and the cart object are passed as props to the Product Listing component and Cart component respectively.

Each product object contains the following properties:

- name: Name of the product. [STRING]
- price: Price of the product. [NUMBER]
- id: Unique ID of the product. (Auto Generated) [NUMBER]
- image: The image URL of the product. [STRING]
- cartQuantity: The quantity of the item in the cart. The default value should be 0. [NUMBER]

Each item in the cart, CartItem, has the following properties:

- id: The ID of the product added to the cart. [NUMBER]
- item: The name of the product added to the cart. [STRING]
- quantity: The quantity of the item in the cart [NUMBER]

The following data-testid attributes are required in the component for the tests to pass:

- Each product item in the Product Listing component should have the data-testid attribute of 'product-item-0', 'product-item-1', and so on.
- Each 'Add to Cart' button should have the data-testid attribute 'btn-item-add'.
- Each 'Increase Quantity' button should have the data-testid attribute 'btn-quantity-add'.
- Each 'Decrease Quantity' button should have the data-testid attribute 'btn-quantity-subtract'.
- Each input to display the cart quantity in the Product Listing component should have the data-testid attribute 'cart-quantity'.
- The table rows <tr> in the Cart component, corresponding to items in the cart, should have the data-testid attribute as 'cart-item-0', 'cart-item-1', and so on.
- The table data <td>, containing the name of the item in the cart, should have the data-testid attribute 'cart-item-name'.
- The table data <td>, containing the quantity of the item in the cart, should have the data-testid attribute 'cart-item-quantity'.

Please note that the component has the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 3

React: Kanban Board

SCORE: 50 points

Easy

React

Front-End Frameworks

Component State Management

Event Handlers

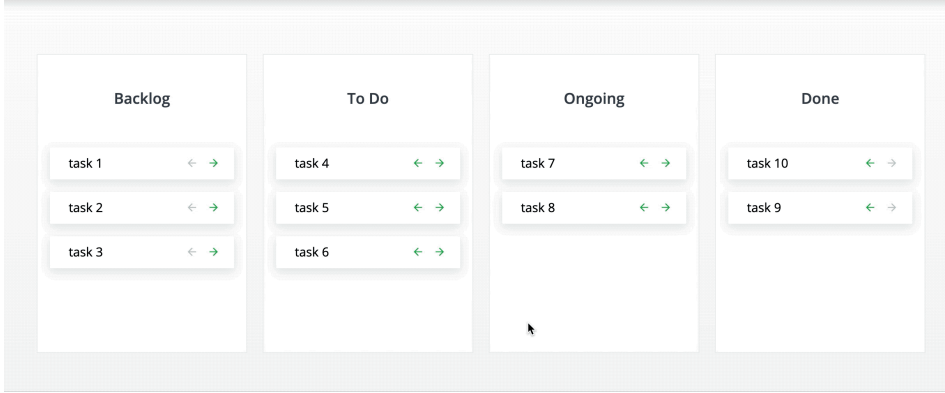
Class Components

JavaScript

Kanban is a popular workflow used in task management, project management, issue tracking, and other similar purposes. The workflow is usually visualized using a [Kanban Board](#).

Create a Kanban Board component with tasks, where each task consists of a name only, as shown below:

[Hide animation](#) [Show animation](#)



The component must have the following functionalities:

- The component board contains 4 stages of tasks in the sequence 'Backlog', 'To Do', 'Ongoing', and 'Done'.
- An array of tasks is passed as a prop to the component.
- In every individual stage, the tasks are rendered as a list `<ul>`, where each task is a single list item `<li>` that displays the name of the task.
- Each task list item has 2 icon buttons on the right:
 1. Back button: This moves the task to the previous stage in the sequence, if any. This button is disabled if the task is in the first stage.
 2. Forward button: This moves the task to the next stage in the sequence, if any. This button is disabled if the task is in the last stage.
- Each task has 2 properties:
 - name: The name of task. This is the unique identification for every task. [STRING]
 - stage: The stage of the task. [NUMBER] (0 represents the 'Backlog' stage, 1 represents the 'To Do' stage, 2 represents the 'Ongoing' stage, and 3 represents the 'Done' stage)

The following data-testid attributes are required in the component for the tests to pass:

- The `<ul>` for the 'Backlog' stage should have the data-testid attribute 'stage-0'.
- The `<ul>` for the 'To Do' stage should have the data-testid attribute 'stage-1'.
- The `<ul>` for the 'Ongoing' stage should have the data-testid attribute 'stage-2'.
- The `<ul>` for the 'Done' stage should have the data-testid attribute 'stage-3'.
- Every `<li>` task should follow these guidelines:
 1. The `<span>` containing the name should have the data-testid attribute 'TASK_NAME-name', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-name'. For the task named 'abc', it should be 'abc-name'.
 2. The back button should have the data-testid attribute 'TASK_NAME-back', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-back'. For the task named 'abc', it should be 'abc-back'.
 3. The forward button should have the data-testid attribute 'TASK_NAME-forward', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-forward'. For the task named 'abc', it should be 'abc-forward'.

Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 4

SCORE: 50 points

React: Timer Component

React

Easy

Front-End Frameworks

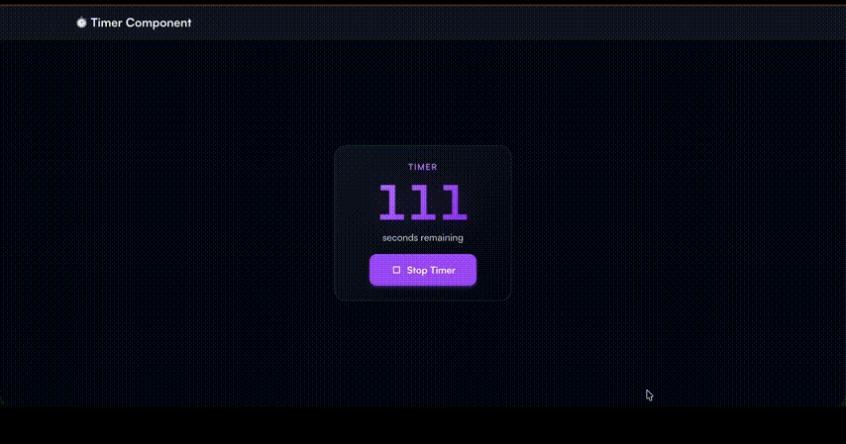
Controlled Component

Class Components

JavaScript

Create a React Timer component that counts down from an initial value, updates every second, and allows the user to stop the timer manually.

[Hide animation](#) [Show animation](#)



Your Users Should Be Able To:

- Pass an initial value to the component through a prop named initial.
- See the timer start automatically when the component mounts, decreasing by 1 every second.
 - Example: if initial = 100, after 1 second it shows 99, then 98, and so on.
- See the timer stop automatically when the value reaches 0.
- Click the Stop Timer button to stop the countdown at its current value.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
Title	app-title
Timer value	timer-value
Stop timer button	stop-button

Please note that components have these *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 5

SCORE: 50 points

React: Navigation Bar Component

React

Front-End Frameworks

Easy

Component State Management

Interviewer Guidelines

Hooks

Node 14

JavaScript

Create a navigation bar component as shown below.

[Hide animation](#) [Show animation](#)

Home

News

Contact

About

HOME PAGE

The component has the following functionalities:

- There are 4 tabs: Home, News, Contact, and About.
- The default selected tab is the 'Home' tab.
- Clicking on a tab renders the relevant content.
 - Clicking on the Home tab renders content 'HOME PAGE'.
 - Clicking on the News tab renders content 'NEWS PAGE'.
 - Clicking on the Contact tab renders content 'CONTACT PAGE'.
 - Clicking on the About tab renders content 'ABOUT PAGE'.
- Since the default selected tab is the 'Home' tab, the default displayed content is 'HOME PAGE'.

Please note that components have some classes and ids for rendering purposes. It also has the following *data-testid* attributes for test cases.

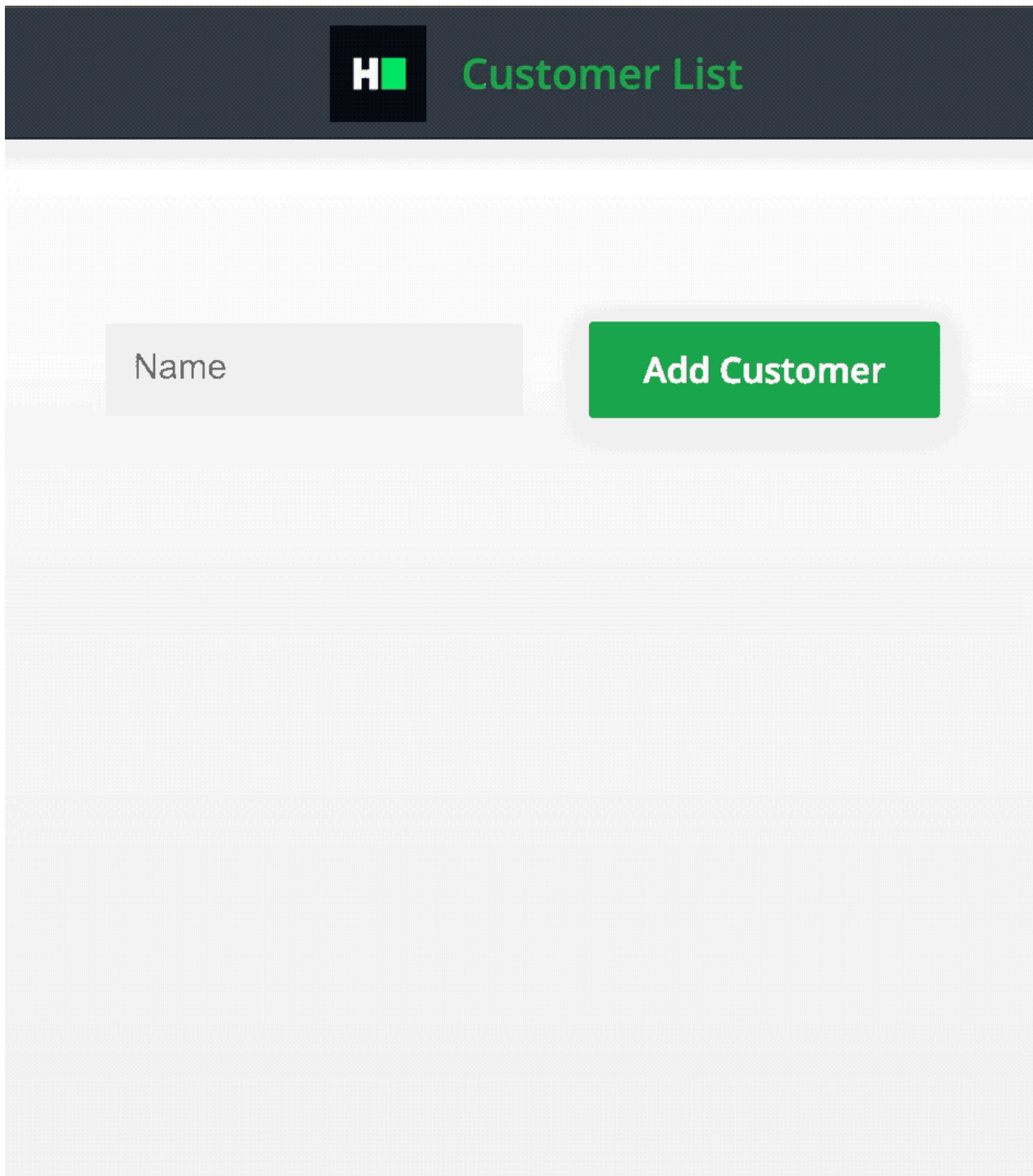
- The parents of 4 tabs have *data-testid* attribute 'navigation-tabs'.
- The content sections have *data-testid* attribute 'tab-content'.

It is not advised to change any of these classes, ids, or *data-testid* attributes.

Question - 1
React: Customer List

Create a customer list component as shown.

[Hide animation](#) [Show animation](#)



- The component must have the following functionalities:
- The input should initially be empty.
 - If no value is entered, clicking on the 'Add Customer' button should not do anything.
 - Clicking on the 'Add Customer' button should add the input value to the list below. For this, add `<li>{input}</li>` to the `<ul data-testid="customer-list">` element.
 - After the value is added to the list, it should clear the value in the input box.
 - Please note that the customer list `<ul>` element should only be rendered if it has at least one customer added, i.e. at least one `<li>` child. When the app is mounted, since no customers are added, the `<ul>` element should not be rendered.
 - All the values added by the button should be rendered in the list below.

The following `data-testid` attributes are required in the component for the tests to pass.

Component	Attribute
Input	<code>app-input</code>
Button	<code>submit-button</code>
Customer list <code></code>	<code>customer-list</code>
List elements	<code>list-item0</code> , <code>list-item1</code> , ...

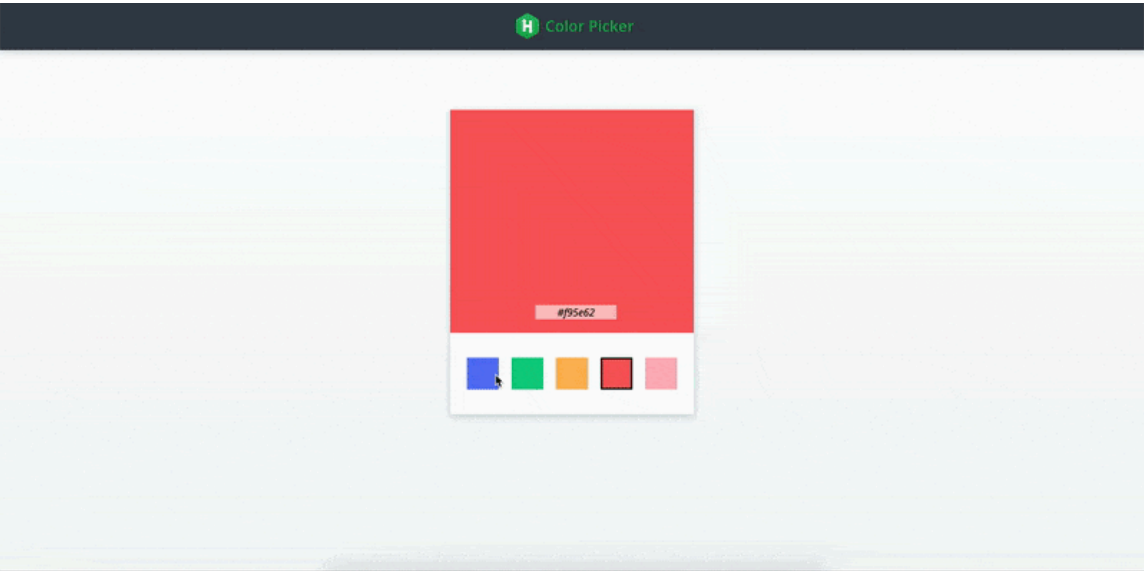
Please note that components have these `data-testid` attributes for test cases, and certain classes and ids for rendering purposes. They should not be changed.

Question - 2

React: Color Picker

Create a color picker component as shown below.

[Hide animation](#) [Show animation](#)



- The component should have the following functionalities:
- The `selectedColor` div shows the chosen color.
 - Initially, the color of this div is set by the `props` property `initialSelectedColor`, a valid CSS color name, e.g., "red" or "#efefef".
 - The color list div contains a list of colors. Clicking any color in this list changes the color of the `selectedColor` div.
 - The `backgroundColor` of the clicked div is the color to set as `backgroundColor` of the `selectedColor` div. The colors in this list are passed as props via the property 'colorPickerOptions' which is an array of strings, where each string is a valid CSS color name.

The following `data-testid` attributes are required in the component for the tests to pass.

Component	Attribute
Title	<i>app-title</i>
<i>selectedColor</i> div	<i>selectedColor</i>
Color list	<i>colorPickerOptions</i>

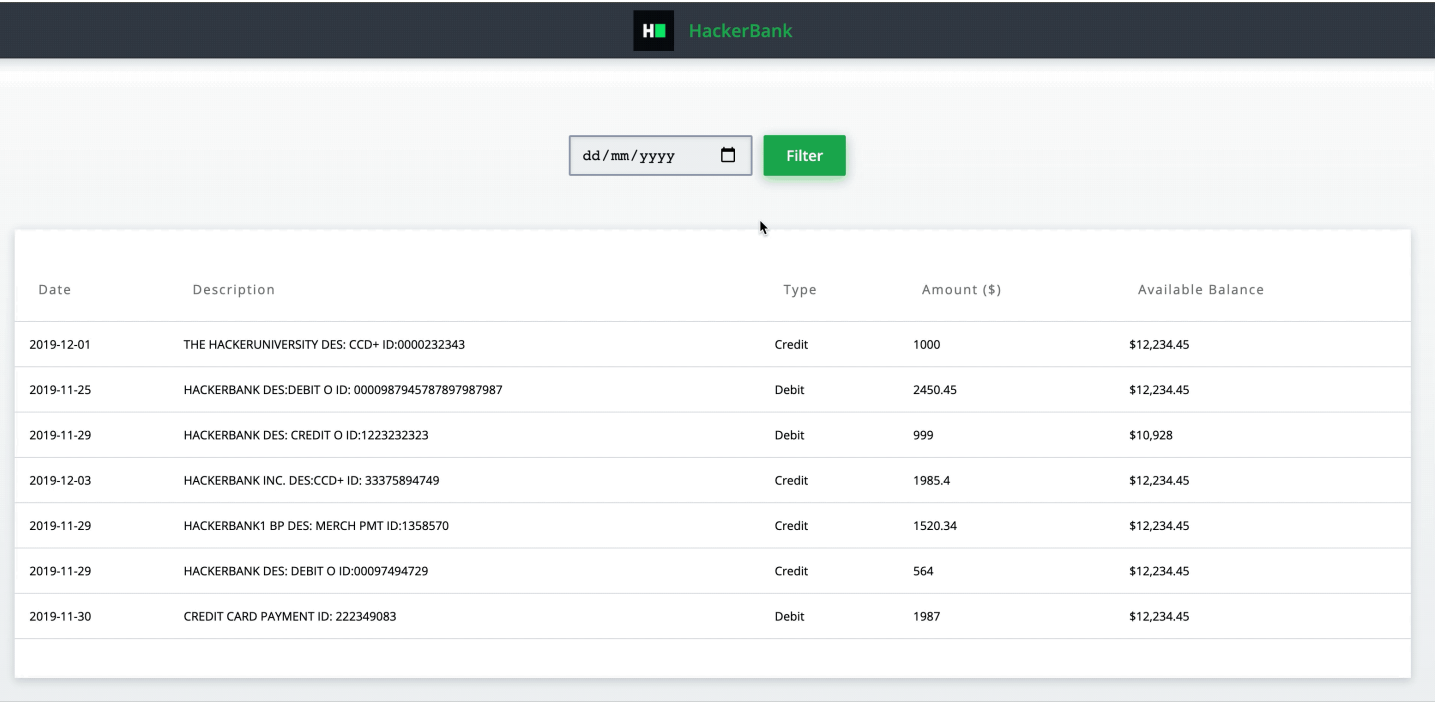
Please note that the colors are set by `backgroundColor` style specifically. Tests rely on this to pass. The component has certain classes and ids for rendering purposes. They should not be changed.

Question - 3

React: HackerBank

Create a banking application as shown below. Some core functionalities have already been implemented, but the application is not complete. Application requirements are given below, and the finished application must pass all of the unit tests.

[Hide animation](#) [Show animation](#)



- Implement the following functionalities:
- All transactions are initially displayed inside the table in the order they are retrieved from the source. The source is passed down as `'txns'` prop to `TransactionTable` component in `App.js`.
 - Picking the date from the date input and pressing the 'Filter' button should display all the records for that date in the table. If no date is chosen, the 'Filter' button should not do anything.
 - Clicking on the 'Amount (\$)' table header should sort the records in ascending order of amount. The behavior is the same for multiple clicks on 'Amount (\$)'.

- Each transaction object contains the following properties :
- *String* date: The date when the transaction took place in the format YYYY-MM-DD.
 - *String* description: The description of the transaction.
 - *Number* type: The type of transaction, where 0 denotes a credit transaction and 1 denotes a debit transaction.
 - *Float* amount: The total amount of the transaction.
 - *String* balance: The balance of the account after the transaction was completed, prefixed with a dollar sign (\$).


```
{
  "date": "2019-12-03",
  "description": "HACKERBANK INC. DES:CCD+ ID: 33375894749",
  "type": 0,
  "amount": 1985.4,
  "balance": "$12,234.45"
}
```

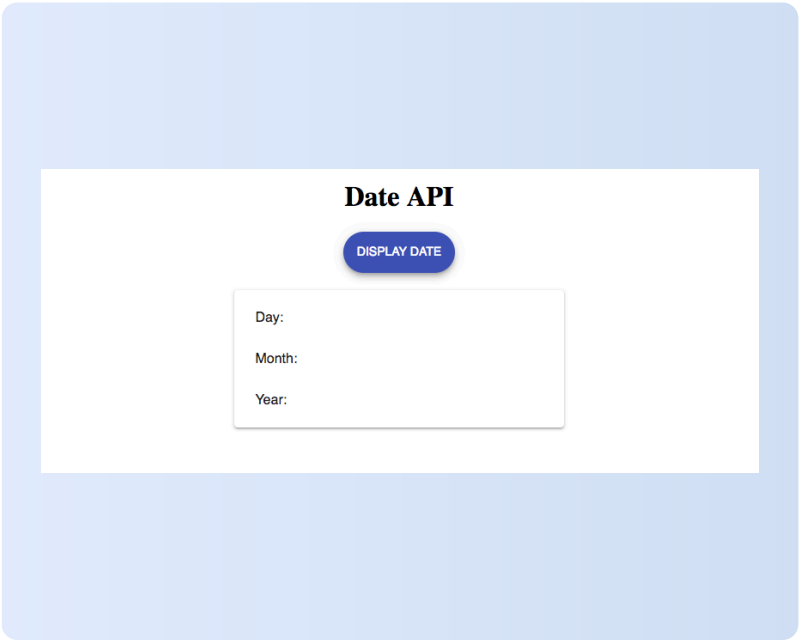
Question - 4

React: Date API

In the given single page React project, a button click prompts an HTTP GET request to an [API endpoint](#) that returns a *date* as its API response. Complete the project so it displays the current day, month, and year after clicking the button. Model the implementation after the instructions below. Certain core React functionalities have already been implemented. Complete the React application in order to pass all the unit tests.

Demo

- [Link to an animated GIF](#)
- Scroll sideways below to explore the screenshots. Clicking each screenshot opens it in a new window.



Application's initial state before clicking the button.

Date API

DISPLAY DATE

Day: 3

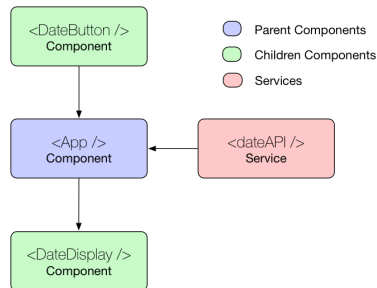
Month: 8

Year: 2018

Display date after clicking the button.

Specifications

- Component flow diagram of the application is as follows:



▼ Framework Specific Instructions

- The project uses React 16 by default. Changing the React version may interfere with tests and is strictly discouraged. You may refer to the React 16 docs [here](#).
- The project uses [create-react-app](#) and [react-scripts](#) to automate serving and testing of the application.
- The project uses [enzyme](#) as a testing framework. Please refrain from changing the test framework or the tests themselves.
- The project uses React Material UI as a design framework. API docs are available [here](#) and can be used as a reference.